Übungsblatt 1: Speicherphysik & RAM-Typen

Mikrocomputertechnik 3 · 32 Orientierungspunkte · ca. 75–90 Minuten

Musterlösung

Aufgabe 1: Speicher für einen Messwertpuffer (5 Punkte)

Ein Messwertpuffer enthält 768 KiB Nutzdaten. Ein KiB sind 1024 Byte. Vergleichen Sie klassische 6T-SRAM- und 1T1C-DRAM-Zellen.

1. Berechnen Sie Byte- und Bitzahl sowie Zelltransistoren beider Varianten. Wie viele Kondensatoren enthält die DRAM-Variante? (3 P)
2. Warum ergibt die Bauteilzählung keine genaue Chipfläche? (1 P)
3. Ein Ein-Port-Speicher soll im selben Takt einen Befehl und einen Messwert liefern. Er bedient höchstens eine Anfrage je Takt. Was muss geschehen? (1 P)

Musterlösung 1

1. $768 \cdot 1024 = 786432$ Byte, 6291456 Bit. SRAM: $6 \cdot 6291456 = 37748736$ Zelltransistoren. DRAM: je 6291456 Transistoren und Kondensatoren. (3 P)
2. Peripherie, Bauteilgrößen und Kondensatorgeometrie fehlen. (1 P)
3. Eine Anfrage muss warten; der Engpass ist der vorgegebene Port. (1 P)

Aufgabe 2: Konfigurationsblock aktualisieren (5 Punkte)

Ein logischer Flashblock enthält die Byte `C6`, `5A`, `FF`, `FF`. Gelöscht ist `FF`; Programmieren erlaubt nur eins nach null, Löschen setzt den gesamten Block auf `FF`.

1. Prüfen Sie getrennt die Änderungen des ersten Bytes nach `84` und nach `CE`. Schreiben Sie die beteiligten Werte binär. (2 P)
2. Beschreiben Sie das Update nach `CE`, wenn `5A` erhalten bleiben muss. (1 P)
3. Zwölf physische Löschzyklen treffen vier gleichartige Blöcke entweder als $12, 0, 0, 0$ oder als $3, 3, 3, 3$. Vergleichen Sie Summe und größte Belastung. (1 P)
4. Welche zusätzliche Systembedingung braucht die Ausführung von Code aus NOR-Flash? (1 P)

Musterlösung 2

1. `C6` = `11000110`, `84` = `10000100`, `CE` = `11001110`. Nach `84` sind nur Eins-nach-null-Änderungen nötig. Nach `CE` ist null nach eins nötig: löschen. (2 P)
2. Benötigte Blockinhalte sichern, Block löschen, `CE` und `5A` wieder programmieren. (1 P)
3. Summe jeweils zwölf; Maximum zwölf beziehungsweise drei. Verschleiß bleibt endlich. (1 P)
4. Ein speicherabbildbarer ausführbarer Leseweg mit passendem Controller und Zugriffsrechten. (1 P)

Aufgabe 3: SRAM mit gesetztem Q lesen und umschreiben (6 Punkte)

Eine klassische 6T-SRAM-Zelle hält anfangs $Q = 1$, $Q_B = 0$. Links steuert Q_B die Transistoren P_Q und N_Q, rechts steuert Q die Transistoren P_QB und N_QB. WL steuert die beiden nMOS-Zugriffstransistoren; BL gehört zu Q, BLB zu QB. pMOS leitet im qualitativen Modell bei Gate null, nMOS bei Gate eins.

1. Nennen Sie die leitenden und sperrenden Invertertransistoren beim Halten. (2 P)
2. Beide Bitleitungen wurden hoch vorgeladen. Welche sinkt beim Lesen leicht? Ordnen Sie Vorladung ausschalten, WL anheben, Auswerten und erneute Vorladung. (2 P)
3. Q soll auf null wechseln. Welche Bitleitungswerte sind vor WL-Anstieg nötig? Wann dürfen die Schreibtreiber freigegeben werden? (2 P)

Musterlösung 3

1. P_Q und N_QB leiten; N_Q und P_QB sperren. WL ist beim Halten niedrig. (2 P)
2. BLB am Nullknoten sinkt. Vorladung aus, WL hoch, Differenz auswerten, WL niedrig, anschließend neu vorladen. Der logische Zellinhalt bleibt erhalten. (2 P)
3. BL null, BLB eins; Daten vor WL bereitstellen und bis nach WL-Abfall halten. Danach Treiber freigeben. (2 P)

Aufgabe 4: DRAM mit anderen Zellkapazitäten (6 Punkte)

Lehrmodell: $C_{\text{cell}} = 24$ fF, $C_{\text{BL}} = 240$ fF, Platte null Volt, Vorladung 0,5 V; Zellzustände 0 oder 1 V. Beim Verbinden gilt Ladungserhaltung, weitere Effekte sind ausgeblendet.

1. Berechnen Sie beide gemeinsamen Spannungen und deren Abstand zur Vorladung. (3 P)
2. Ordnen Sie Restore, WL-Absenken und Precharge. (1 P)
3. Ein Adressmodell hat acht Banken mit je 64 Zeilen, 32 Spalten und vier Byte je Spaltenelement. Feldreihenfolge von hoch nach niedrig: Bank, Zeile, Spalte, Byte. Bestimmen Sie Kapazität, Feldbreiten und die Adresse für Bank fünf, Zeile elf, Spalte neunzehn, Byte zwei. (2 P)

Musterlösung 4

1. $V = (C_{\text{cell}}V_{\text{cell}} + C_{\text{BL}}V_{\text{pre}})/(C_{\text{cell}} + C_{\text{BL}})$. Hoch: $144/264 \approx 0{,}54545$ V; niedrig: $120/264 \approx 0{,}45455$ V. Abstände etwa $\pm 45{,}45$ mV. (3 P)
2. Restore bei hoher WL, danach WL niedrig, anschließend BL vorladen. (1 P)
3. $8 \cdot 64 \cdot 32 \cdot 4 = 65536$ Byte = 64 KiB. Felder 3/6/5/2 Bit; $5 \cdot 8192 + 11 \cdot 128 + 19 \cdot 4 + 2 = 42446 =$ `0xA5CE`. (2 P)

Aufgabe 5: Speicherplan eines Datenloggers (6 Punkte)

Byteadressiertes Modell: Flash ab `0x00200000`, 256 KiB; SRAM ab `0x21000000`, 96 KiB. Ein ausgerichteter 32-Bit-Zugriff benötigt vier Byte und einen durch vier teilbaren Start.

1. Bestimmen Sie exklusive Bereichsenden und letzte Byteadressen. (1 P)
2. Prüfen Sie Zugriffe ab `0x21017FFC`, `0x21017FFF` und `0x21018000`. (1 P)
3. Ohne Padding: Code 36 KiB, Konstanten 6 KiB, initialisierte veränderbare Daten 512 Byte, Nullinitialisierung 8 KiB, Stack 3 KiB. Initialwerte liegen zusätzlich im Flash, Nullbytes nicht. Berechnen Sie Bedarf und freien Platz in beiden Speichern. (2 P)
4. Wer kopiert Initialwerte und setzt den Nullbereich zurück? Erzwingt C-const eine Flashplatzierung? (2 P)

Musterlösung 5

1. Flashende `0x00240000`, letztes Byte `0x0023FFFF`; SRAM-Ende `0x21018000`, letztes Byte `0x21017FFF`. (1 P)
2. `0x21017FFC`: gültig und ausgerichtet; `0x21017FFF`: zu lang und unausgerichtet; `0x21018000`: ausgerichtet, außerhalb. (1 P)
3. Flash: 42,5 KiB, frei 213,5 KiB. SRAM: 11,5 KiB, frei 84,5 KiB. (2 P)
4. Startcode führt Kopieren und Nullsetzen aus, der Linker legt Bereiche fest. const allein erzwingt keinen physischen Flashort. (2 P)

Aufgabe 6: Ringpuffer mit fünf Plätzen (4 Punkte)

Fünf nutzbare Plätze, anfangs Leseindex r, Schreibindex w und count jeweils null. Erfolgreiche Indizes wachsen modulo fünf; abgewiesene Vorgänge ändern nichts. Folge: K, L, M, N einlegen; ein Element entnehmen; P und Q einlegen.

1. Notieren Sie r, w und count nach jedem Vorgang sowie die abschließende Entnahmereihenfolge. (2 P)
2. Was geschieht beim anschließenden Einlegen von R? Warum reicht r gleich w nicht als Leerheitsprüfung? (1 P)
3. Wählen Sie Speicherprinzipien für dauerhaftes Programmbild und häufig geänderte Zwischensummen. (1 P)

Musterlösung 6

1. Zustände $(r, w, count)$: $(0,1,1)$, $(0,2,2)$, $(0,3,3)$, $(0,4,4)$, nach Entnahme K $(1,4,3)$, danach $(1,0,4)$ und $(1,1,5)$. Reihenfolge L, M, N, P, Q. (2 P)
2. R wird ohne Änderung abgewiesen. Gleiche Indizes können bei count null oder fünf auftreten. (1 P)
3. Nichtflüchtiger Speicher etwa Flash für das Programmbild, SRAM für Zwischensummen. (1 P)

Optional: am Rechner prüfen

Erweitern Sie `src/lab01_memory.c` um die Flashfälle aus Aufgabe 2 und die Adressgrenzen aus Aufgabe 5. Ergänzen Sie einen Ring mit fünf Plätzen und prüfen Sie Aufgabe 6. Die bisherigen Testfälle der Datei sind keine Lösung dieser neuen Aufgaben.

```
gcc -std=c11 -O2 -Wall -Wextra -Werror -pedantic \
    labs/src/lab01_memory.c -o /tmp/lab01
/tmp/lab01
```

Aufruf vom Repository-Hauptordner. Geprüft wird ein logisches Hostmodell.